COMP3821/9801

Workshop Week 8

Divide and Conquer

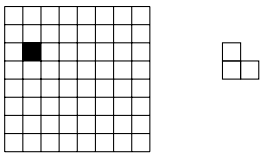
Copyright Disclaimer. *Reproducing, publishing, posting, distributing, translating this document is an infringement of copyright and will be referred to UNSW Student Conduct and Integrity for action.*

Guided Problem

The demonstrators will solve the following problems. For the rest of the workshop, work in small groups to solve the remaining workshop exercises. There are optional problems at the end of this document for further practice.

Guided Problem

You are given an $n \times n$ board with one of its cells missing (i.e. the board has a hole). You are given a supply of L-trominoes, each covering three cells as pictured below.

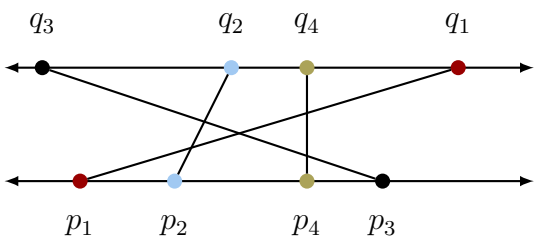


Describe and analyse a polynomial-time algorithm (in terms of n) to cover the entire board with the L-shaped trominoes.

Note. *You may assume that n is a power of two and you know where the missing cell is in advance.*

Guided Problem

You are given two lists of n points, one list $P = [p_1, \ldots, p_n]$ lies on the line $y = 0$ and the other list $Q = [q_1, \ldots, q_n]$ lies on the line $y = 1$. We construct n line segments by connecting p_i to q_i for each $i = 1, \ldots, n$. You may assume that the numbers in P are distinct and the numbers in Q are also distinct. Design an $O(n \log n)$ algorithm to return the number of intersections between every pair of distinct line segments.
For example, the following instance

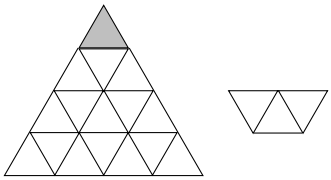


should return 5 since there are five intersections.

Workshop Exercises

1. Let n be a power of two. An equilateral triangle is partitioned into smaller equilateral triangles by parallel lines dividing each of its sides into $n > 1$ equal segments. The topmost equilateral triangle is chopped off (indicated as shaded in the diagram below). We want to tile the remaining equilateral triangles with trapezoids, each of which is composed of three equilateral triangles. A valid tiling has two properties:

   - Every equilateral triangle must be covered by at least one trapezoid.
   - No equilateral triangle may be tiled by more than one trapezoid.

   In this example, the sides are divided into $n = 4$ segments.

   Describe and analyse a polynomial-time algorithm (in terms of n) to tile the grid with a valid tiling, using only trapezoids.

   Note. *A tiling always exists.*

2. Let A be a sequence of n distinct integers. We say that a pair (i, j) of indices is a *major inversion* if and only if $i < j$ and $A[i] > 2025 \cdot A[j]$. Describe and analyse a polynomial-time algorithm that counts the number of major inversions of A.

3. In this problem, we will design a divide and conquer algorithm to compute $n!$ for a given integer n. Note that $n!$ has $\Theta(\log(n!)) = \Theta(n \log n)$ bits.

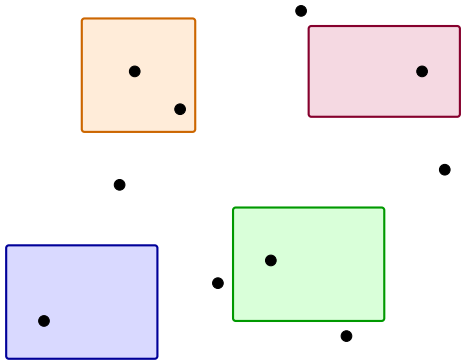
   For this problem, we will use the fact that, using the Karatsuba trick, we can multiply two k-bit integers in time $O(k^{1.59})$ and that we can multiply a k-bit integer with an ℓ-bit integer in time $O(\ell \cdot k^{0.59})$ with $\ell > k$.

   (a) Consider the naive iterative solution: for $i = 1$ to n, multiply the current result with i. Show that the algorithm computes $n!$ in time $O(n^2 \log^{1.59} n)$, where $\log^{1.59} n$ is $(\log n)^{1.59}$.

   (b) Describe and analyse an $O(n^{1.59} \cdot \log^{1.59} n)$ algorithm that computes $n!$.

   Hint. *Try solving the more general problem of computing $(M+1)(M+2)\ldots(M+n)$ for any given integer M, n; we can retrieve the solution by setting $M = 0$.*

4. **[Not assessable]** We are given a set R of n_1 rectangles in the plane; assume that the rectangles are axis aligned. In other words, the sides of the rectangles are parallel to the x and y axes. We also know that the rectangles do not overlap. We are also given a set P of n_2 points in the plane. For every point $p \in P$, we want to find the rectangle $r_p \in R$ that contains p.

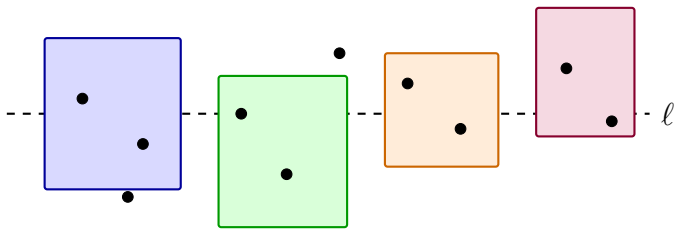


Note. *If no rectangle contains p, then we return "undefined" for p.*

Let $n = n_1 + n_2$. We can define the location of a rectangle by identifying its bottom-left and top-right corners. Additionally, point p_i is represented by a point $P[i]$. Formally, we are given three arrays:

- $B = [(x_1, y_1), \ldots, (x_{n_2}, y_{n_2})]$, where $B[i]$ represents the bottom-left point of rectangle r_i.
- $T = [(x_1, y_1), \ldots, (x_{n_2}, y_{n_2})]$, where $T[i]$ represents the top-right point of rectangle r_i.
- $P = [(x_1, y_1), \ldots, (x_{n_1}, y_{n_1})]$, where $P[i]$ represents the location of point p_i.

(a) We first consider the special case where all of the rectangles intersect a given horizontal line ℓ. Describe and analyse a polynomial-time algorithm to solve the problem under this special case.



(b) Now, describe and analyse a polynomial-time algorithm to solve the original problem.

Hint. *You may assume that a recurrence of the form $T(n) = 2T(n/2) + O(n \log n)$ yields complexity $O(n \log^2 n)$ without proof.*

Note. *If time permits, you are encouraged to discuss how to solve this problem as a group since it introduces divide and conquer in a novel manner.*

Optional Exercises

These exercises will not contain solutions. We encourage that you post on the forums to discuss the approaches for these problems. Additionally, you may find the hints helpful.

5. Let $A[1..n]$ be a sorted array of n *distinct* integers. Some of these integers may be positive, negative, or zero.

   (a) Describe and analyse an algorithm to decide if there exist some index i such that $A[i] = i$ in $O(\log n)$ time.

   (b) Now, suppose that we know that $A[1] > 0$. Answer the same problem in $O(1)$ time.

6. Let $A[1..n]$ be an array on n integers such that $A[1] \geq A[2]$ and $A[n-1] \leq A[n]$. An element $A[i]$ of A is said to be a *local minimum* if both $A[i-1] \geq A[i]$ and $A[i] \leq A[i+1]$ conditions are true. For example, the following array has three local minima, highlighted below.

3	1	4	1	5	9	2	6
	↑		↑			↑	

   (a) Given the constraints that $A[1] \geq A[2]$ and $A[n-1] \leq A[n]$, show that a local minimum exists in A.

   (b) Describe and analyse an $O(\log n)$ algorithm to find one of these local minimum elements. If the array has multiple elements, your algorithm just needs to return one of them.

7. Let $A[1..n]$ and $B[1..n]$ be two *sorted* arrays containing n distinct integers each. Describe and analyse an $O(\log n)$ algorithm that finds the median of $A \cup B$, where $A \cup B$ is the array obtained by merging arrays A and B together.

8. Let $A[1..n]$ be an array of n integers. We are given an additional integer x, and our task is to decide whether x appears somewhere in A.

   (a) Suppose you know that A is a sorted array but it has been shifted by k units to the right. For example, if $k = 4$ and the sorted array was $[1, 2, 3, 5, 6, 7]$, then $A = [5, 6, 7, 1, 2, 3]$. In this scenario, suppose we are given k in advance. Describe and analyse an $O(\log n)$ algorithm that decides whether x appears somewhere in A.

   Note. *You are given, as input, an array that is promised to be shifted by k units to the right, where k is known.*

   (b) Suppose that we do not know what k is. Describe and analyse an $O(\log n)$ algorithm that decides whether x appears somewhere in A.

   Hint. *If we can find k in $O(\log n)$ time, then we can just use the previous algorithm.*

9. You are given an $n \times n$ array A, which you do not have direct access to. In other words, you do not know the values of each cell of A in advance. You are given that the array satisfies the following two properties.

   - The entries of A are distinct; that is, no distinct pair of entries are the same.
   - The largest value in each row is non-decreasing.

   Your task is to compute an array $M[1..n]$ of n elements such that $M[i]$ is the column number of the largest value of the ith row. Design an algorithm that

computes M, using only $O(n \log n)$ number of queries to A. For example, consider the following matrix.

10	5	8	3
1	9	7	6
-3	4	-1	0
-10	-9	-8	2

Your algorithm should return $M = [1, 2, 2, 4]$ since $A[1, 1], A[2, 2], A[3, 2], A[4, 4]$ are the maximum values in their respective rows.

10. Let $A[1..m]$ and $B[1..n]$ be two sorted arrays containing m and n distinct integers respectively. Given A, B, and an integer k, Describe and analyse an $O(\log(m+n))$ to find the kth smallest element of $A \cup B$, where $A \cup B$ is the array obtained by merging arrays A and B together.

11. In the field of computer graphics, a *blit* (***bl**ock **t**ransfer*) is a low-level operation, which takes in a rectangular block of pixels from a pixel map and copies the block to some location in a different pixel map. You may assume that each blit operation takes constant-time.

    A blit can be defined by the top left and bottom right cells; that is, copy the subgrid bounded by $(x_1, y_1), (x_1 + m, y_1 + n)$ on one grid to the subgrid bounded by $(x_2, y_2), (x_2 + m, y_2 + n)$ on a second grid. This copies the first subgrid into the second subgrid componentwise, maintaining the relative positions of the cells; overwriting the contents of the second grid. The top left cell of the first subgrid *must* go into the top left cell of the second subgrid.

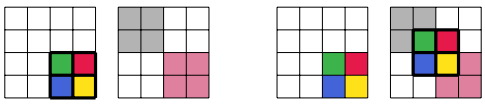


Figure 1: *Performing a blit from a 2×2 subgrid to another 2×2 subgrid, each in distinct grids.*

Note the following:

- only part of the grid gets moved,
- the blitted subgrid doesn't necessarily maintain its positioning, and
- the relative positioning of the cells in the blitted subgrid remains the same.

Let n be a power of two. In this problem, we are given an $n \times n$ pixel map and our goal is to rotate the map 90° clockwise.

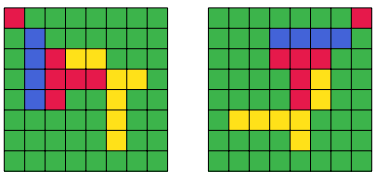


Figure 2: *Rotating the pixel map 90° clockwise.*

Describe and analyse an $O(n^2)$ algorithm that outputs a sequence of blit operations to rotate a pixel map 90° clockwise. You do not need to explicitly give

the sequence; instead, show how the sequence is constructed and collected. You may assume that $\Theta(n^2)$ temporary space is available for you to transfer and cache subgrids.

12. In a game of Among Us, there are n players, k of which are imposters. At every round, we choose a subset of these players to call a meeting and discuss among themselves. A meeting is *won* by the players if it is found that no imposter was called to the meeting and the meeting is *lost* if at least one of the imposters was called to the meeting. At the end of each meeting, we only know of the outcome and not which of the players in the meeting was an imposter.

    The goal is to design a strategy to identify all of the imposters using only $O(k \log(n/k))$ meetings.

    (a) Suppose that we partition the set of n players into m disjoint groups and send each of these groups into a separate meeting. Show that at least $m - k$ meetings are won.

    (b) Using the previous result, design a recursive procedure to identify all k imposters.

    (c) Show that the previous procedure takes $O(k \log(n/k))$ many meetings to identify all k imposters.

13. You are given an array $A[1..n]$ of n positive integers, where $A[i]$ represents the value of a stock on day i. Your goal is to buy the stock on a particular day and then sell the stock at a later day to maximise your profit.

    Note. *You may assume that n is a power of two.*

    (a) Firstly, suppose that you want to buy the stock on the first $n/2$ days and then sell it on the last $n/2$ days. Describe and analyse an $O(n)$ algorithm to find the best pair of days to buy and sell the stock.

    (b) Describe and analyse an $O(n \log n)$ algorithm to find the best pair of days to buy and sell the stock.

    (c) Modify the previous algorithm to obtain an $O(n)$ algorithm.

14. An important problem in algorithm theory is to construct an efficient algorithm to find the median of a given dataset. Algorithms such as QuickSort rely on fast median-finding algorithms for its efficiency. On the other hand, *median filtering* plays a crucial role in image processing. In this problem, we will use a divide and conquer approach to devise a tight algorithm for median-finding. We formalise the problem as follows.

    - MedianFinding: Given an array $A[1..n]$ of n integers, find the median of A (i.e. the middle value[1] when A is sorted).

    In this problem, we will describe a deterministic algorithm that finds the median of an array in deterministic $O(n)$ time, which matches its lower bound – see part (a).

    (a) Informally, explain why any *deterministic* median-finding algorithm needs to spend $\Omega(n)$ number of comparisons to find the median of a given array.

    (b) Firstly, describe a simple $O(n \log n)$ algorithm to find the median of an array on n integers. Therefore, we need to find a method to beat this bound.

[1] Or one of the middle values when n is even.

(c) Assume we pick an element p of A at random. Let L denote the set of all elements that are smaller than p, and let R denote the set of all elements that are larger than p. Describe a recursive algorithm to find the median of A. If the pivot point p is chosen poorly, show that the algorithm can degenerate to worst-case running time $O(n^2)$ time. Therefore, we need to find ways of picking a good pivot.

(d) Consider the following algorithm.

- Partition the array A into groups of five; if the last group has less than five elements, that is okay.
- Sort each group and compute the median of each group.
- Recursively, compute the median of the median list using the same procedure as above.

Note that the median obtained after this procedure terminates is only an *approximation* to the actual median. Use the median obtained by the above procedure to compute the median of A in $O(n)$ time. Hence, show that we can solve the median-finding problem in $O(n)$ time.

(e) If we allow randomisation as we did in part (c), describe an algorithm to find the kth smallest element with expected running time $O(n)$. This problem is known as the *order statistic* problem.

Note. *The median-finding problem is a special case of the order statistic problem, where* $k = \lfloor n/2 \rfloor$.

15. Let $\mathcal{I} = \{I_1, \ldots, I_n\}$ be a set of n intervals. Each interval I_i is specified by its two end points: a_i and b_i with $a_i \leq b_i$. Two intervals I_i and I_j overlap if there exist a point x such that $x \in I_i$ and $x \in I_j$. In other words, $x \in [a_i, b_i]$ and $x \in [a_j, b_j]$. Finally, the length of the overlap is given by the longest interval that is shared between I_i and I_j. Formally, we can express this length as

$$L(i,j) = \max\{0, \min\{b_i, b_j\} - \max\{a_i, a_j\}\}.$$

Given n intervals, Describe and analyse an $O(n \log n)$ algorithm that returns the pair of intervals I_i and I_j that maximises $L(i,j)$. Your input is two arrays $A[1..n]$ and $B[1..n]$ such that $A[i]$ specifies the beginning of interval I_i and $B[i]$ specifies the end of interval I_i.

16. Let $x_1, \ldots, x_n \in \mathbb{R}^2$ be a set of n points in the two-dimensional plane. We measure the distance $d(x_i, x_j)$ by the Euclidean distance; that is,

$$d(x_i, x_j) = \sqrt{(x_{i,1} - x_{j,1})^2 + (x_{i,2} - x_{j,2})^2}.$$

Assume you have access to this distance function as a black box oracle. Describe and analyse an $O(n \log n)$ algorithm to return the pair of points x_i, x_j whose distance is minimised. Note that brute forcing the problem gives an $O(n^2)$ algorithm.

17. Let $G = (V, E)$ be an undirected graph. Unfortunately, you do not know what G looks like. Therefore, you cannot directly access G. However, you are given a blackbox function f that takes two *distinct* subsets S, T and returns TRUE if and only if there is at least one edge between a vertex in S and a vertex in T.

Given two subsets $S, T \subseteq V$ of vertices, describe an algorithm that computes the number of edges with one endpoint in S and one endpoint in T, using only $O(|E| \log |V|)$[2] queries to f.

Note. *The actual running time of the algorithm might exceed than the number of queries. But we must use $O(|E| \log |V|)$ number of queries.*

18. Consider an $m \times n$ chessboard. A knight is a special chess piece that can only move in an L shape. A *Knight's tour* is a path through the chessboard via a sequence of knight moves such that every square of the chessboard is visited exactly once. We say that a chessboard is *knight-tourable* if there exist a knight's tour through the chessboard.

    Let $n \geq 6$ be an even integer. Describe and analyse an $O(n^2)$ algorithm to show that any $(n + 2) \times n$ chessboard is knight-tourable.

19. Let T be a binary tree whose nodes store *distinct* integers. We say that T is a *binary search tree* if either T is an empty tree, or T satisfies the following properties:

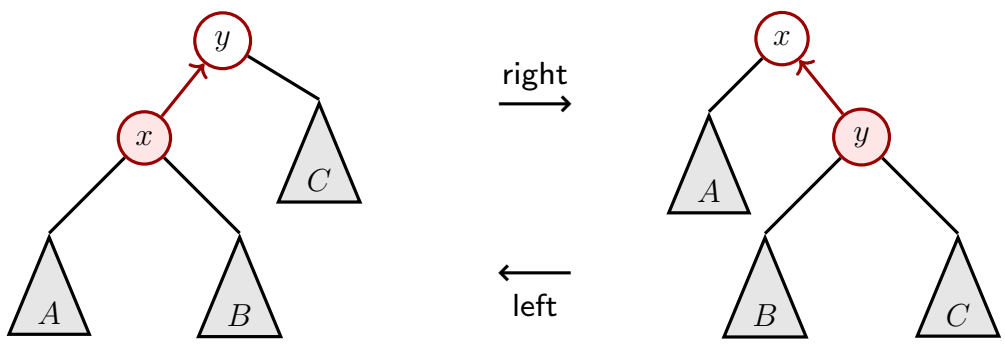
    - The left and right subtrees of T are binary search trees themselves.
    - The values in the left subtree of T are smaller than the value at the root of T.
    - The values in the right subtree of T are larger than the value at the root of T.

    Your task is to transform an arbitrary binary tree T into a binary search tree, but you can only modify T with rotations and swaps. In other words, you cannot perform the following operations:

    - Insert new nodes or delete nodes from T.
    - Directly modify parent or child pointers.
    - Change the value stored at any of the nodes.

    In case you have forgotten, here's a picture of the rotation and swap operations.

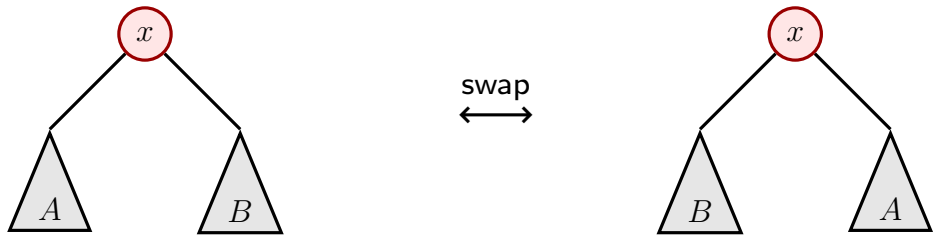
    - $\textsc{Rotate}(x, d)$. Given a node x and a direction $d \in \{L, R\}$, rotate x in the direction of d.



Rotating to the right: $\textsc{Rotate}(x, R)$. Rotating to the left: $\textsc{Rotate}(y, L)$.

- $\textsc{Swap}(x)$. Given a node x, swap the left and right subtrees of x.

[2] A tighter analysis shows that one can get worst-case upper bound $O(|E| \log(|V|^2/|E|))$. With randomisation, we can get $(1 + \epsilon)$-approximations with only $O(\text{polylog } n)$ calls to f.



Given an arbitrary binary tree T with n nodes, describe an algorithm that transforms T into a binary search tree using at most $O(n \log n)$ rotation and swap operations.

Hints

5. Consider the array B where $B[i] = A[i] - i$. Try showing that B is monotonic. For the second part, how many elements do we need to check to decide if such an index exists?
6. You just need to find a feasible local minima, not find the global minimum element. Therefore, you should recurse on the half that you know definitely has a local minima.
7. Consider splitting A and B in half; how do we know which halves of A and B to discard?
8. If we know the pivot, what do we know about the structure of the subarrays to the left and right of the pivot point?
9. Try using the same ideas as some of the earlier problems; which elements can we discard?
10. This is a generalisation of Problem 7. Try using the same ideas.
11. Try using similar ideas as the guided problem.
12. If at least $m - k$ meetings are won, then you know that these meetings do NOT contain an imposter. We can eliminate these players. You want to choose a specific value for m so that you can reduce the search space.
13. The best pair of days is the pair of days (d_1, d_2) such that d_1 comes before d_2 with d_1 being minimum and d_2 being maximum; your goal is to find this efficiently.
14. See median of medians.
15. Split the array into two halves; we can recursively find the pair of intervals in both halves. How do you find the optimal solution that crosses the splitting point in $O(n)$ time.
16. How do you ensure that you only need $O(n)$ time in the merge step?
17. For any subsets S, T of vertices in G, let $m(S, T)$ denote the number of edges between S and T. Consider partitioning S into disjoint sets S_1, S_2 and partitioning T into disjoint sets T_1, T_2. Convince yourself that

$$m(S, T) = m(S_1, T_1) + m(S_1, T_2) + m(S_2, T_1) + m(S_2, T_2).$$

18. Consider dividing the board into four parts, what kind of moves must be present for a knight tour to exist?
19. You can turn your binary tree into a doubly linked list using rotations. Then try to sort it using the same mechanism as merge sort but only using rotations and swaps – what do rotations represent and what do swaps represent?